

Category	Method	Usage Example with fake()	Sample Output
Basic Data	name	fake()->name()	John Doe
	firstName	fake()->firstName()	John
	lastName	fake()->lastName()	Doe
	email	fake()->email()	john.doe@example.com
	password	fake()->password()	a1b2c3d4
Address	address	fake()->address()	123 Main Street, Springfield, IL 62704
	city	fake()->city()	Springfield
	country	fake()->country()	United States
	postcode	fake()->postcode()	62704
	streetAddress	fake()->streetAddress()	123 Main Street
Phone Numbers	phoneNumber	fake()->phoneNumber()	(555) 123-4567
	e164PhoneNumber	fake()->e164PhoneNumber()	+15551234567
Dates & Times	date	fake()->date()	2024-12-20
	dateTime	fake()->dateTime()	2024-12-20 15:23:45
	dateTimeBetween	fake()->dateTimeBetween('1 year', 'now')	Random date between last year and now
	time	fake()->time()	15:23:45
Text	text	fake()->text()	Lorem ipsum dolor sit amet, consectetur...
	sentence	fake()->sentence()	The quick brown fox jumps over the lazy dog.
	word	fake()->word()	Random
Numbers	randomDigit	fake()->randomDigit()	7
	randomDigitNotNull	fake()->randomDigitNotNull()	4
	randomNumber	fake()->randomNumber(5, true)	23456

	randomFloat	fake()->randomFloat(2, 0, 100)	42.58
Internet	url	fake()->url()	http://example.com
	domainName	fake()->domainName()	example.com
	ipv4	fake()->ipv4()	192.168.1.1
	ipv6	fake()->ipv6()	fe80::1
Commerce	company	fake()->company()	Acme Corp
	jobTitle	fake()->jobTitle()	Software Engineer
	currencyCode	fake()->currencyCode()	USD
Images	imageUrl	fake()->imageUrl(640, 480, 'cats')	URL for a 640x480 cat image
	image	fake()->image('path/to/dir', 640, 480, 'nature', false)	Downloads an image to the specified directory
Miscellaneous	uuid	fake()->uuid()	123e4567-e89b-12d3-a456-426614174000
	unique	fake()->unique()->email()	Unique email like unique_email@example.com

Migration : php artisan make:migration create_products_table

```
<?php
// Migration for the products table
use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

class CreateProductsTable extends Migration
{
    public function up(): void
    {
        Schema::create('products', function (Blueprint $table) {
            $table->id();
            $table->string('intitule');
            $table->decimal('prix', 8, 2);
            $table->string('image')->nullable(); // Path to the uploaded image
            $table->timestamps();
        });
    }

    public function down(): void
    {
        Schema::dropIfExists('products');
    }
}
```

Factory

```
// Factory for generating fake data
namespace Database\Factories;

use App\Models\Product;
use Illuminate\Database\Eloquent\Factories\Factory;
use Illuminate\Support\Str;

class ProductFactory extends Factory
{
    protected $model = Product::class;

    public function definition(): array
    {
        return [
            'intitule' => $this->faker->words(3, true),
            'prix' => $this->faker->randomFloat(2, 10, 1000),
            'image' => null, // Images need to be handled specifically
        ];
    }
}
```

Seeder

```
namespace Database\Seeders;

use Illuminate\Database\Seeder;
use App\Models\Product;

class ProductSeeder extends Seeder
{
    public function run(): void
    {
        Product::factory(10)->create();
    }
}
```

Model

Php artisan make:model Product

```
// Model for Product
namespace App\Models;

use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;

class Product extends Model
{
    use HasFactory;

    protected $fillable = [
        'intitule',
        'prix',
        'image',
    ];
}
```

Ressource Controller

Php artisan make:controller ProductController -r

Php artisan make:model Product -mcrfs

```
// Resource Controller for Product
namespace App\Http\Controllers;

use App\Models\Product;
use Illuminate\Http\Request;
use Illuminate\Support\Facades\Storage;

class ProductController extends Controller
{
    public function index()
    {
        $products = Product::all();
        return view('products.index', compact('products'));
    }

    public function create()
    {
        return view('products.create');
    }

    public function store(Request $request)
    {
        $request->validate([
            'intitule' => 'required|string|max:255',
            'prix' => 'required|numeric',
            'image' => 'nullable|image|mimes:jpeg,png,jpg,gif|max:2048',
        ]);

        $path = null;
        if ($request->hasFile('image')) {
            $path = $request->file('image')->store('products', 'public');
        }

        Product::create([
            'intitule' => $request->input('intitule'),
            'prix' => $request->input('prix'),
            'image' => $path,
        ]);

        return redirect()->route('products.index')->with('success', 'Product created successfully!');
    }

    public function show(Product $product)
    {
        return view('products.show', compact('product'));
    }

    public function edit(Product $product)
    {
        return view('products.edit', compact('product'));
    }
}
```

```

public function update(Request $request, Product $product)
{
    $request->validate([
        'intitule' => 'required|string|max:255',
        'prix' => 'required|numeric',
        'image' => 'nullable|image|mimes:jpeg,png,jpg,gif|max:2048',
    ]);

    $path = $product->image;
    if ($request->hasFile('image')) {
        if ($product->image) {
            Storage::disk('public')->delete($product->image);
        }
        $path = $request->file('image')->store('products', 'public');
    }

    $product->update([
        'intitule' => $request->input('intitule'),
        'prix' => $request->input('prix'),
        'image' => $path,
    ]);

    return redirect()->route('products.index')->with('success', 'Product updated successfully!');
}

public function destroy(Product $product)
{
    if ($product->image) {
        Storage::disk('public')->delete($product->image);
    }

    $product->delete();

    return redirect()->route('products.index')->with('success', 'Product deleted successfully!');
}
}

```

Routes

```

// Web routes for products
use App\Http\Controllers\ProductController;

Route::resource('products', ProductController::class);

```

Url	verb	Action Controller	Route name	
/products	Get	Index	Products.index	Route::get('/products',[ProductController::class,'index'])->name('products.index')
/products	Post	Store	Products.store	Route::post('/products',[ProductController::class,'store'])->name('products.store')
/products/{id}	Get	Show	Products.show	
/products/create	Get	Create	Products.create	
/products/{id}/edit	Get	Edit	Products.edit	
/products/{id}	Put/patch	Update	Products.update	
/products/{id}	Delete	Destroy	Products.destroy	

View Index

```
// Blade views
// "resources/views/products/index.blade.php"
@extends('layouts.app')
@section('content')
    <a href="{{ route('products.create') }}">Create Product</a>
    <ul>
        @foreach($products as $product)
            <li>{{ $product->intitule }} - {{ $product->prix }}
                intitule }}" width="100">
                <a href="{{ route('products.edit', $product) }}">Edit</a>
                <form action="{{ route('products.destroy', $product) }}" method="POST">
                    @csrf
                    @method('DELETE')
                    <button type="submit">Delete</button>
                </form>
            </li>
        @endforeach
    </ul>
@endsection
```

View Create

```
// "resources/views/products/create.blade.php"
@extends('layouts.app')
@section('content')
    <form action="{{ route('products.store') }}" method="POST" enctype="multipart/form-data">
        @csrf
        <label for="intitule">Intitule:</label>
        <input type="text" name="intitule" id="intitule" required>

        <label for="prix">Prix:</label>
        <input type="number" name="prix" id="prix" step="0.01" required>

        <label for="image">Image:</label>
        <input type="file" name="image" id="image">

        <button type="submit">Create</button>
    </form>
@endsection
```

View Edit

```
// "resources/views/products/edit.blade.php"
@extends('layouts.app')
@section('content')
    <form action="{{ route('products.update', $product) }}" method="POST" enctype="multipart/form-data">
        @csrf
        @method('PUT')

        <label for="intitule">Intitule:</label>
        <input type="text" name="intitule" id="intitule" value="{{ $product->intitule }}" required>

        <label for="prix">Prix:</label>
        <input type="number" name="prix" id="prix" step="0.01" value="{{ $product->prix }}" required>

        <label for="image">Image:</label>
        <input type="file" name="image" id="image">
        @if($product->image)
            intitule }}" width="100">
        @endif

        <button type="submit">Update</button>
    </form>
@endsection
```

NB. Vous devez taper la commande : `php artisan storage:link`

Exercices d'application

Dans un pays on veut gérer les habitants d'une ville donnée, En utilisant Laravel créer une application qui répond aux besoins en suivant les questions .

- Ville (id ,ville , nombreHabitant)
 - Habitant (id , cin , nom ,prenom , #ville_id, photo)
1. Créer le modele Ville avec migration
 2. Créer le modele Habitant avec migration
 3. Gérer la relation one to many au niveau des migrations et des modèles (hasMany , belongsTo)
 4. Créer le controlleur de ressource HabitantController
 5. Coder toutes les actions du controlleur
 6. Créer la liste des routes
 7. Créer les vues : Index , Create , Edit et Show
 - a. Au niveau de la vue Index : filtrer par ville